

微信小程序抓包实战教程

Proxifier+Fiddler+Burp Suite三件套配置与HTTPS解密全流程

做安全测试、接口调试的人，基本都栽过微信小程序抓包的坑。

浏览器 开个代理就能抓的HTTPS，放到小程序里直接失效。系统代理挂得好好的，Burp里一片空白，小程序该怎么加载还怎么加载，流量根本不往代理走。

不是你工具用得不对，是微信小程序 从底层就绕开了系统全局代理。PC端的小程序跑在独立的WeChatAppEx.exe进程里，既不读IE代理，也不读系统代理设置，普通的全局代理工具根本碰不到它的流量。

0x01 为什么普通抓包工具碰不到小程序流量

PC端微信3.9版本之后，所有小程序都剥离到独立的WeChatAppEx.exe进程运行。这个进程的网络栈做了定制，不会读取Windows 系统的代理设置，也不会走浏览器的代理配置。

你在Windows设置里改了全局代理，或者给浏览器装了SwitchyOmega，对这个进程完全无效。它的流量直接从网卡出去，绕过所有系统级代理。

这就是绝大多数人抓包失败的根本原因：你代理加在了系统层，小程序进程根本不鸟你。

要抓到它的流量，必须下沉到进程级强制代理——直接把微信相关进程的所有网络请求，硬生生劫持到代理服务器上。Proxifier干的就是这件事。

微信做这个设计初衷是安全隔离。小程序和微信主进程分拆之后，单个小程序崩溃不会带崩整个微信，同时独立进程的网络栈也能做更严格的风控，避免第三方工具随意篡改流量。但对于合法的开发调试、安全测试来说，这个设计直接拉高了抓包门槛。

不止PC端，移动端微信的小程序同样有独立的渲染进程，加上安卓7.0之后的证书信任机制收紧、小程序自身的SSL Pinning校验，层层叠加下来，新手第一次碰基本都是全军覆没。

0x02 三件套分工与完整流量链路

很多人问，能不能直接Proxifier连Burp？能不能只用Fiddler？

能，但不好用。三个工具搭配是目前PC端小程序抓包稳定性最高、扩展性最强的方案，各有不可替代的作用。

Proxifier：进程级流量劫持核心。强制把WeChatAppEx.exe的所有流量拽到代理链路里，解决「流量根本进不来」的问题。

Fiddler：兼容中转层。专门适配Windows平台的WebView TLS握手，处理微信的特殊证书逻辑，过滤冗余流量，再把干净的业务请求转发给Burp。直接用Proxifier连Burp，会出现大量TLS握手失败、证书报错，小程序直接白屏。

Burp Suite：安全分析主力。拦截篡改、重放爆破、加密参数分析，所有深度测试都在这一层做。完整的流量走向是：

weChatAppEx.exe 进程 → Proxifier 强制劫持 → Fiddler (127.0.0.1:8888) HTTPS解密 → Burp Suite (127.0.0.1:8080) 安全测试 → 公网服务器

从左到右依次为微信小程序进程、Proxifier进程代理层、Fiddler中转解密层、Burp Suite分析层、目标服务器，用箭头标注流量走向，每层标注对应监听端口与核心功能

整个链路的端口固定死，别乱改：

Fiddler监听 127.0.0.1:8888
Burp监听 127.0.0.1:8080

端口错一个，整条链路直接断。

为什么中间要多一层Fiddler？微信小程序的WebView基于定制版Chromium内核，TLS握手的扩展字段、加密套件优先级和普通浏览器不一样，Burp原生的SSL握手逻辑适配性很差，经常出现握手失败、连接重置。

Fiddler对Windows平台的系统证书、WebView兼容性做了深度优化，先由它完成第一次TLS握手和解密，再用标准HTTP格式转发给Burp，稳定性直接拉满。相当于找了个适配性最好的翻译官，先把小程序的特殊协议转成通用格式，再交给Burp做深度分析。

0x03 前置准备：工具版本与环境要求

先把工具备齐，版本尽量对齐，避免奇奇怪怪的兼容问题。

工具清单

Proxifier v4.0 及以上（Windows平台专属，macOS可以用Proxifier for Mac或者Surge替代，配置逻辑一致）
Fiddler Classic 5.0+（免费版足够，别用Fiddler Everywhere，配置逻辑和脚本语法完全不一样）
Burp Suite Community / Professional 2023+ 版本，社区版功能足够基础抓包使用，专业版支持Intruder无限爆破
PC版微信 3.9.0 及以上，打开任务管理器确认存在weChatAppEx.exe进程

环境前提

windows 10 / 11 系统，所有工具全部右键「以管理员身份运行」。进程级流量劫持需要系统权限，普通用户权限会劫持失败
临时关闭360安全卫士、火绒、电脑管家等流量防护软件。这类软件的网络防护会判定进程代理为恶意行为，直接拦截转发链路
工具启动顺序严格遵守：先开Burp → 再开Fiddler → 最后开Proxifier。关闭顺序反过来，不然会有流量漏走或者形成死循环
配置过程中不要开VPN、游戏加速器类工具，会修改系统路由表，干扰Proxifier的流量匹配规则

0x04 Burp Suite 基础配置与证书部署

Burp是整条链路的终点，也是最终做安全分析的核心工具。先把它的监听和证书配好，再往前搭前面的链路。

4.1 配置本地代理监听

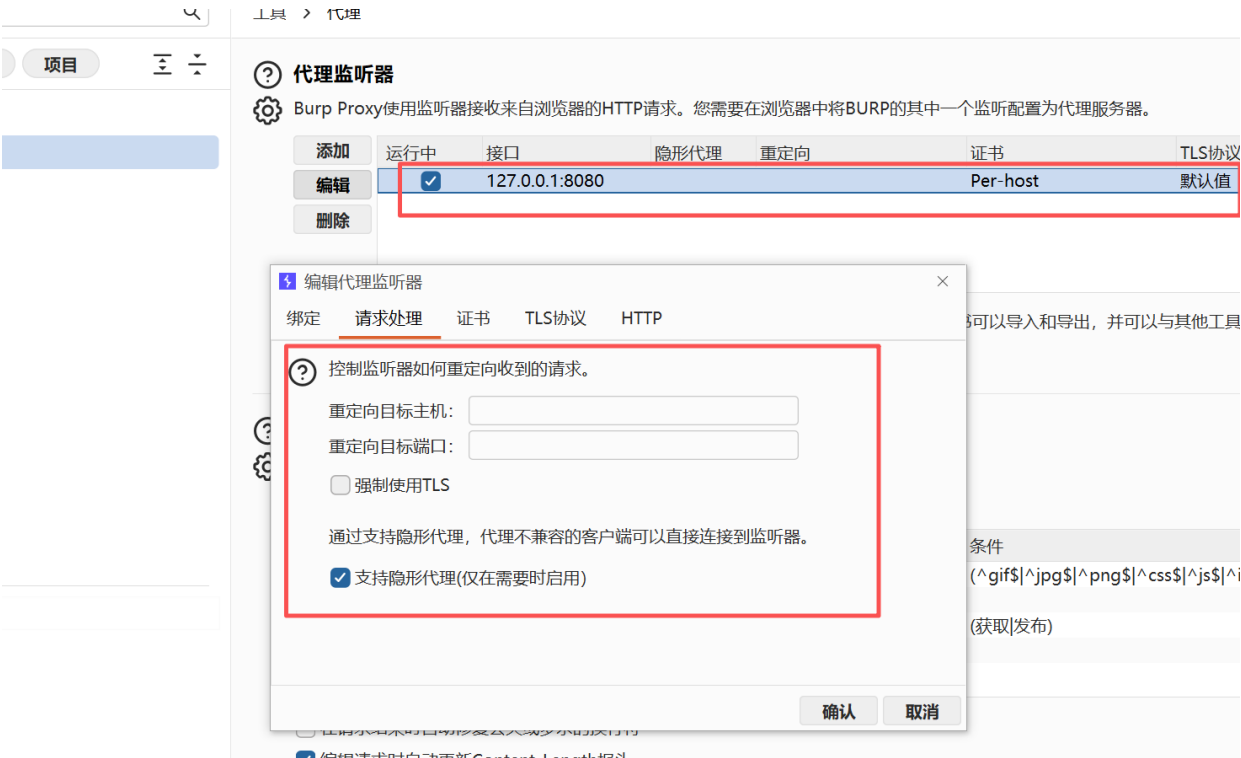
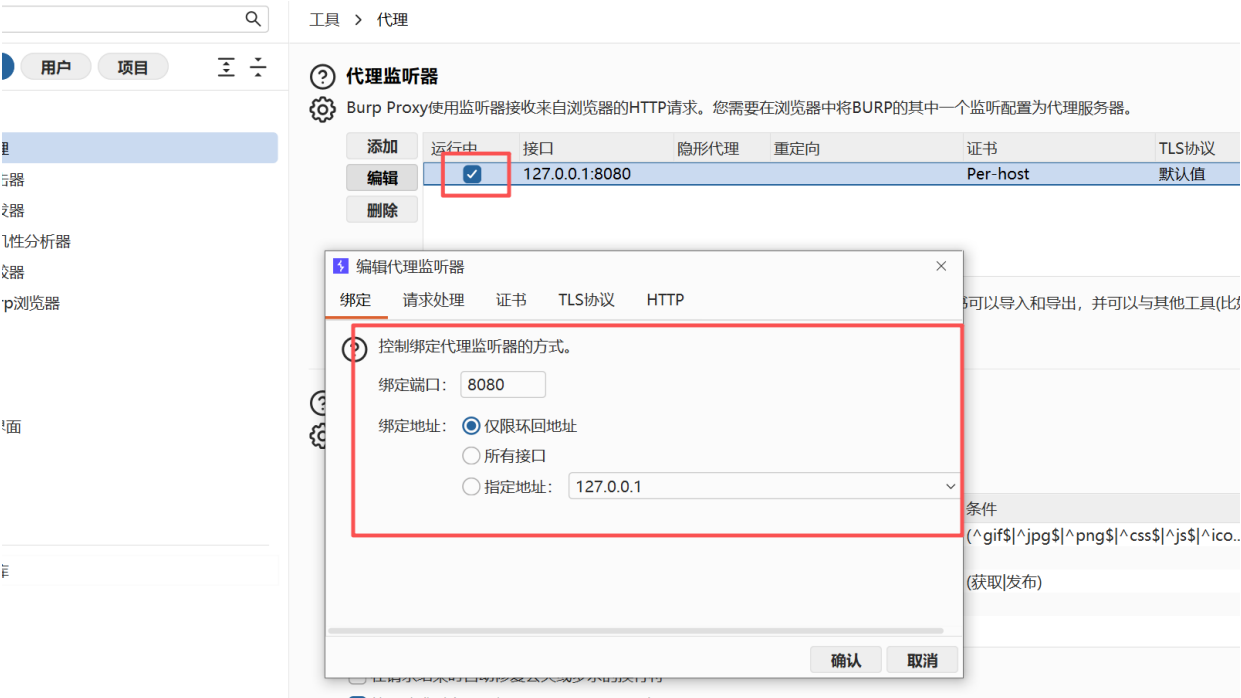
打开Burp Suite，切到Proxy模块，点Settings，找到Proxy Listeners。
点Add新增一条监听规则：

Bind to port 填 8080

Bind to address 选 Loopback only (127.0.0.1)

勾选 Support invisible proxying（透明代理支持）

点OK保存，确认监听状态显示绿色Running。



透明代理这个选项必须勾。小程序进程发的不是标准代理格式的HTTP请求，而是原生的TCP请求，不开启透明代理，Burp接不住这些流量，会直接丢弃。很多人配完发现Burp收不到流量，十有八九是漏了这个选项。

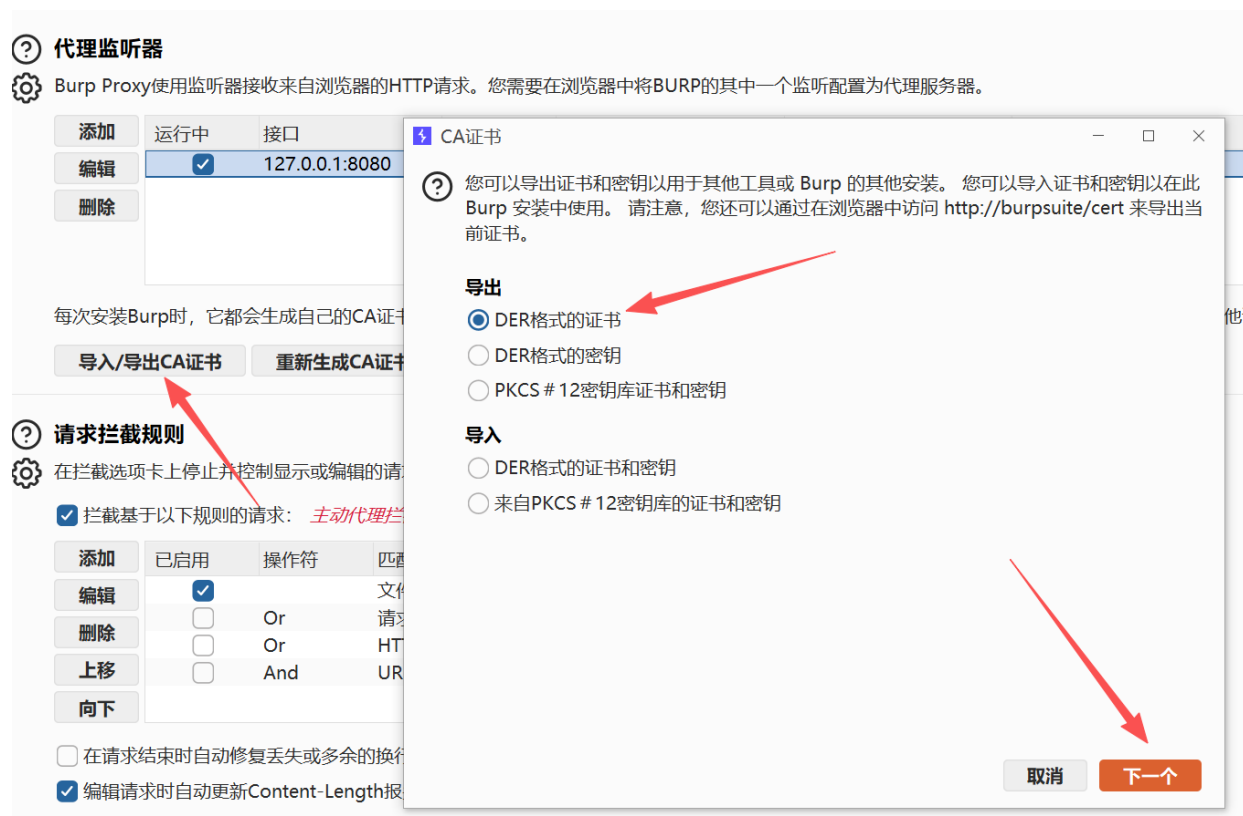
如果后续要接手机端抓包，把监听地址改成All interfaces，临时开放局域网访问即可。

4.2 导出根证书并安装到系统

HTTPS解密的核心是信任中间人证书，这一步装错位置，后面全白搭。

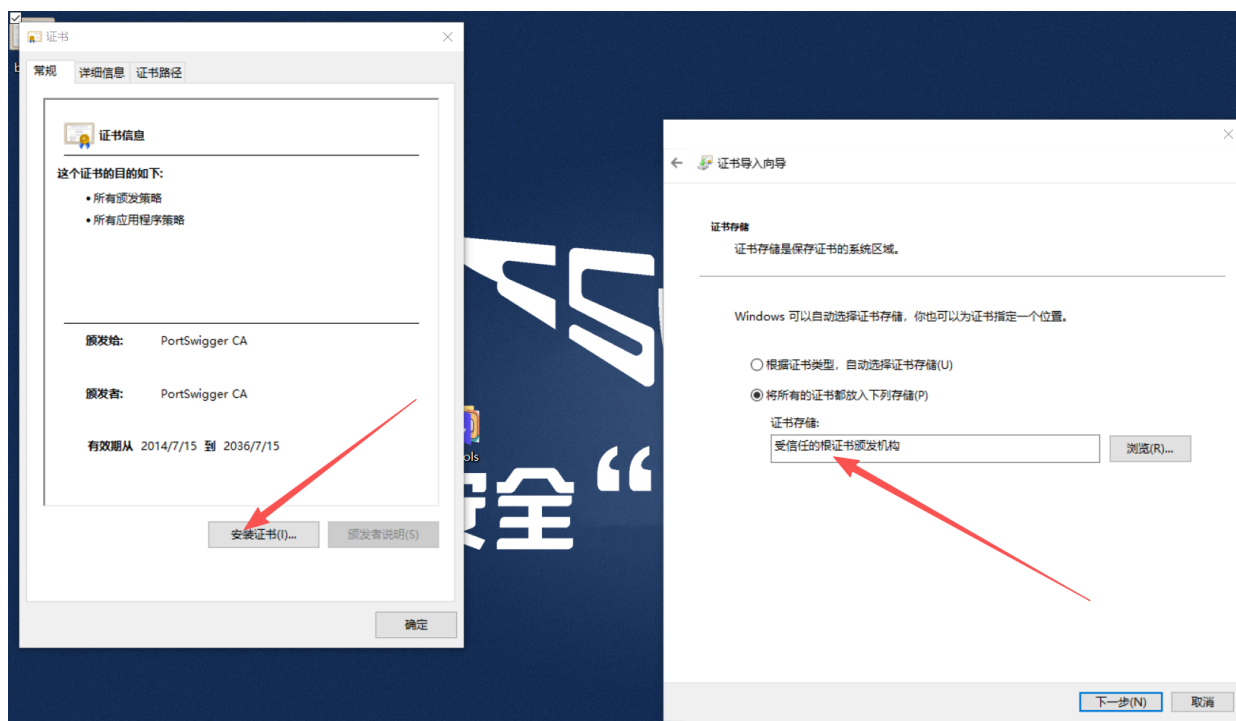
还是在Proxy Settings页面，切到CA Certificate标签，点Export CA Certificate。

导出格式选 DER format (.cer)，文件命名BurpCA.cer，存到桌面就行。



找到桌面上的证书文件，双击打开：

点「安装证书」，存储位置选 本地计算机，别选当前用户
下一步，选「将所有的证书都放入下列存储」
点「浏览」，找到「受信任的根证书颁发机构」，选中确定
一路下一步，提示导入成功。



很多教程只说安装证书，不说装哪个位置。微信小程序进程运行在系统权限上下文，只会读取本地计算机的证书存储，装在当前用户的证书完全无效，这是HTTPS解密失败排名第一的原因。

装完证书重启Burp，让配置生效。后续Fiddler的证书也会自动安装到相同位置，两者互不冲突。

Burp的HTTP history页面可以提前配置好过滤规则，省得后面被冗余流量淹没。点Filter按钮，勾选Hide CSS、Hide images、Hide other static files，把静态资源全部隐藏，只显示API接口请求。后续还可以按域名过滤，只保留目标业务的接口，排查效率会高很多。

0x05 Fiddler 中转配置

Fiddler承担两个核心作用：**承接Proxifier的HTTP代理流量，完成第一次HTTPS解密；再把解密后的请求转发给Burp做深度分析。**

5.1 开启HTTPS解密与基础监听

打开Fiddler Classic，顶部菜单栏 点Tools → Options。

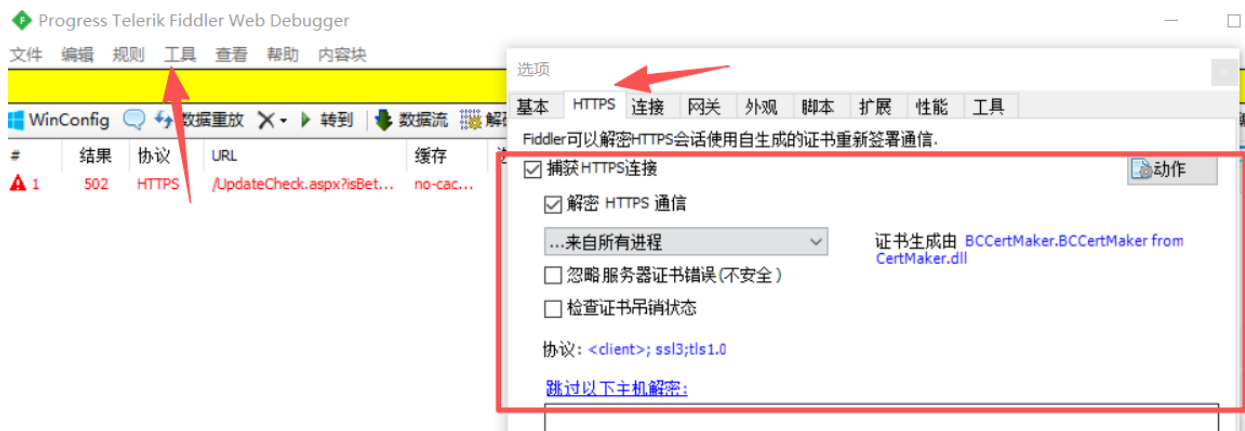
先切换到HTTPS标签页：

勾选 Decrypt HTTPS traffic

旁边的下拉选 All Processes

弹出的证书信任提示全部点Yes，让Fiddler把自己的根证书也装到系统里

协议：<client>;ssl3;tls1.0;tls1.2



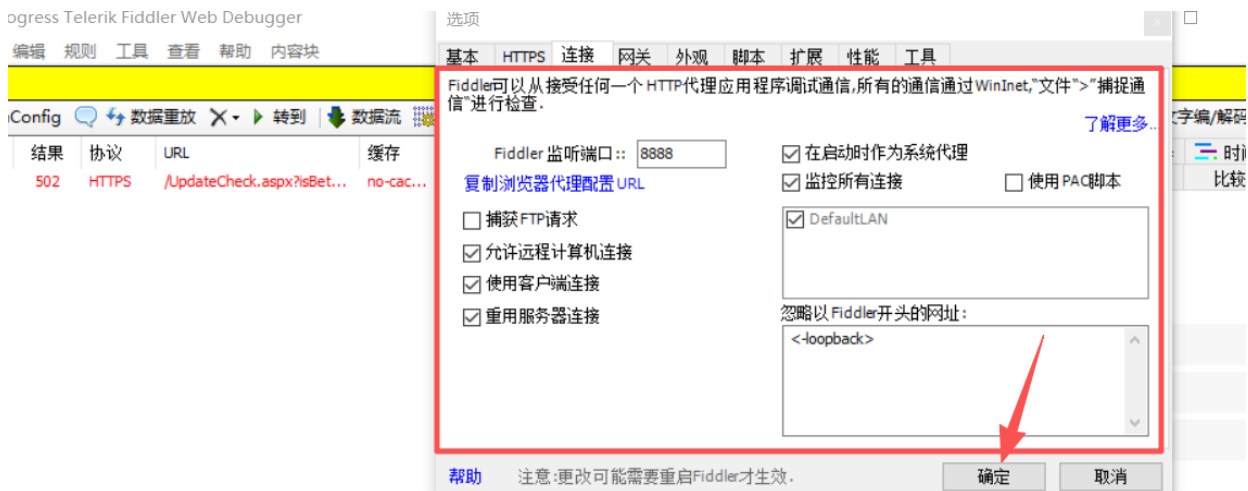
再切换到Connections标签页:

Fiddler listens on port 填 8888

勾选 Allow remote computers to connect

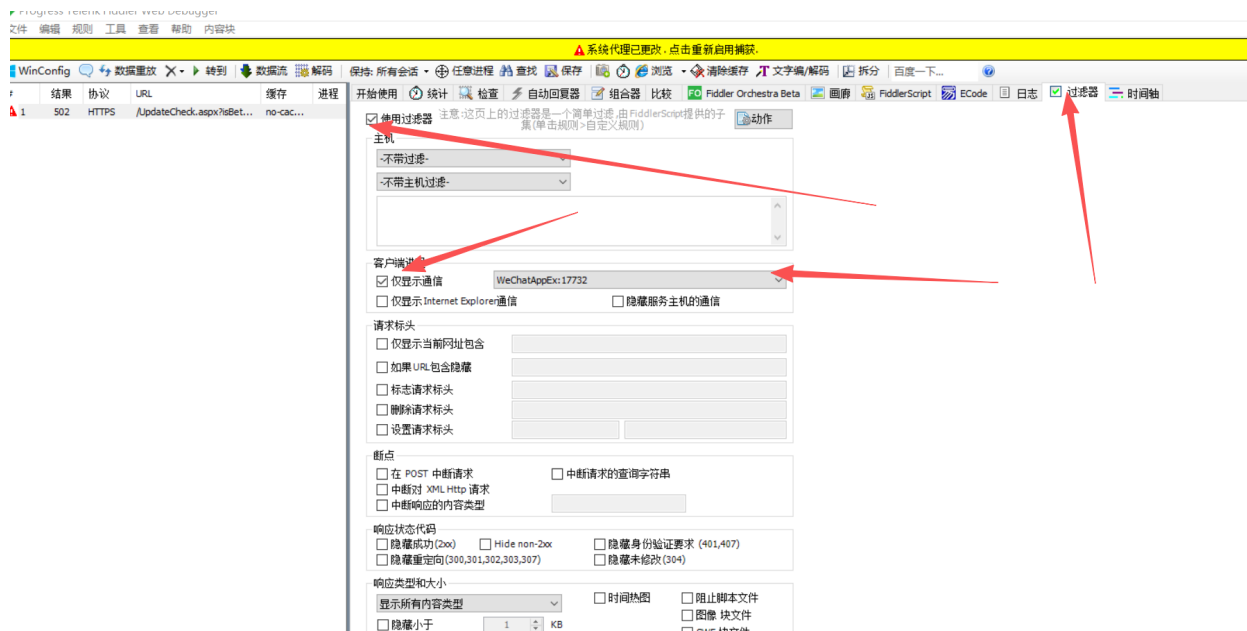
Bypass proxy for servers that start with 输入框里填 <loopback>

点OK保存, 重启Fiddler。



<loopback> 这条规则是放行本地回环地址的流量, 避免本地服务的请求也被代理, 减少不必要的干扰。如果后续要接手机抓包, 这里不用改, 手机直接连8888端口即可。

日常调试的时候, 可以在Fiddler右侧Filters标签里开启进程过滤。勾选Use Filters, Processes选Show only traffic from, 下拉选WeChatAppEx.exe, 这样会话列表里就只有小程序的流量, 不会被微信主进程的心跳包、更新请求刷屏。



5.2 配置上游转发：把流量导进Burp

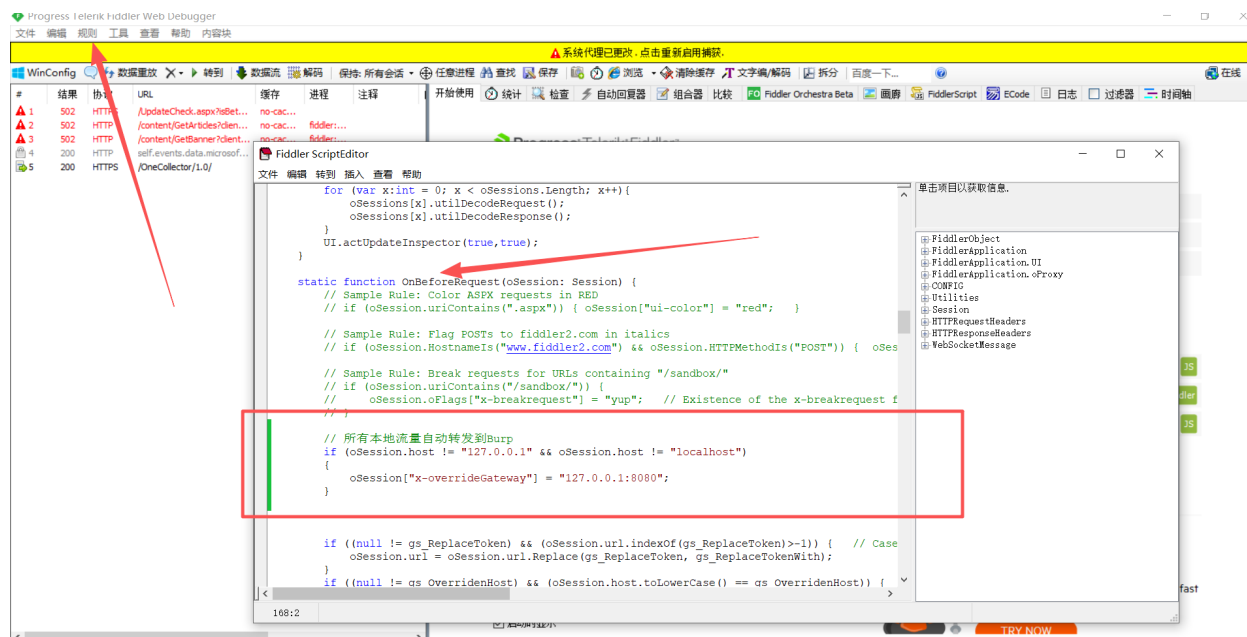
Fiddler默认直接把流量发去公网，我们要让它把非本地的请求全部转发给Burp，由Burp做后续的安全分析。

顶部菜单栏点Rules → Customize Rules，打开Fiddler ScriptEditor脚本编辑器。

找到OnBeforeRequest函数，在函数的最开头插入下面这段代码：

```
// 所有本地流量自动转发到Burp
if (oSession.host != "127.0.0.1" && oSession.host != "localhost")
{
    oSession["x-overrideGateway"] = "127.0.0.1:8080";
}
```

按Ctrl+S保存脚本，Fiddler会自动加载新规则，不用重启。



这段代码的逻辑很简单：只要请求的地址不是本地回环，就全部转发到127.0.0.1的8080端口，也就是Burp的监听地址。本地流量不转发，避免本地服务之间的调用形成死循环。

别直接改Fiddler的全局上游代理。全局代理会把所有流量都导去Burp，包括Fiddler自己的心跳、更新请求，很容易出问题。用脚本精确控制转发范围，稳定性高很多。

如果只想转发特定域名的流量到Burp，其他流量直接走公网，可以把判断条件改成域名匹配。比如只转发api.example.com的请求：

```
// 指定域名全部转发到Burp Suite 8080端口
if (oSession.host.ToLower().Contains("baidu.com")) {
    oSession["x-overrideGateway"] = "127.0.0.1:8080";
}
```

这种模式适合只针对单个业务做测试，其他微信基础接口直接放行，减少Burp的无效流量。

0x06 Proxifier 进程级强制代理配置

这是整套配置最核心的一步，规则顺序错了直接翻车。Proxifier的代理规则是**从上到下匹配**的，命中第一条就不会往下走，顺序错了要么抓不到流量，要么直接死循环断网。

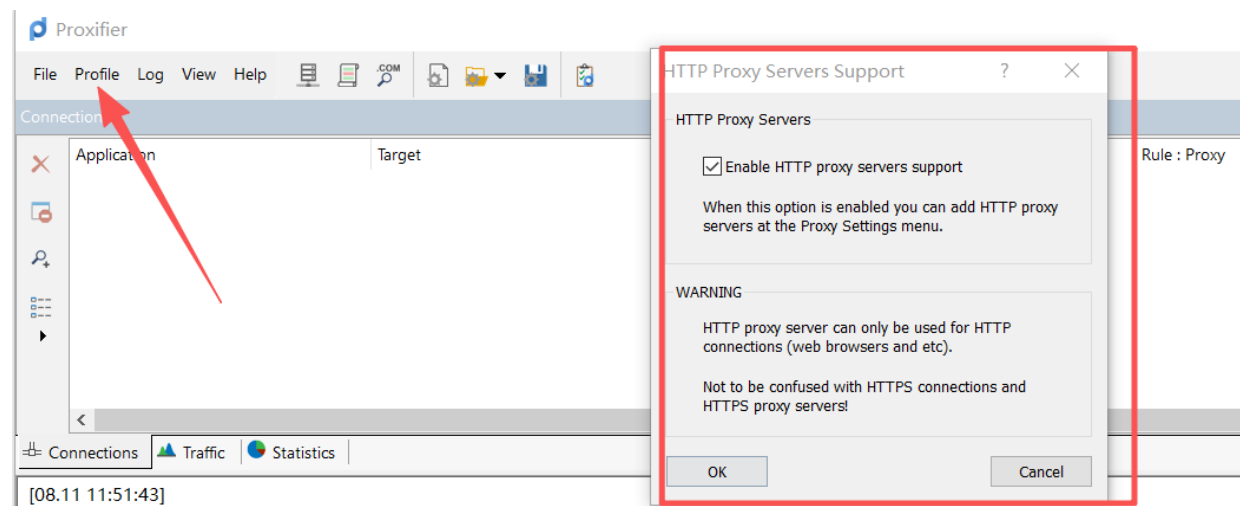
6.1 开启HTTP代理支持

Proxifier默认只支持SOCKS代理，我们要用它连Fiddler的HTTP代理，得先手动开启HTTP代理支持。

顶部菜单栏点 配置文件 → 高级 → HTTP代理服务器。

勾选「启用HTTP代理服务器支持」，点确定。

这一步漏掉，会导致后面加代理服务器的时候一直报错连不上，记死了先开这个开关。



6.2 添加Fiddler代理节点

顶部菜单栏点 配置文件 → 代理服务器 → 点Add。

填参数：

Address: 127.0.0.1

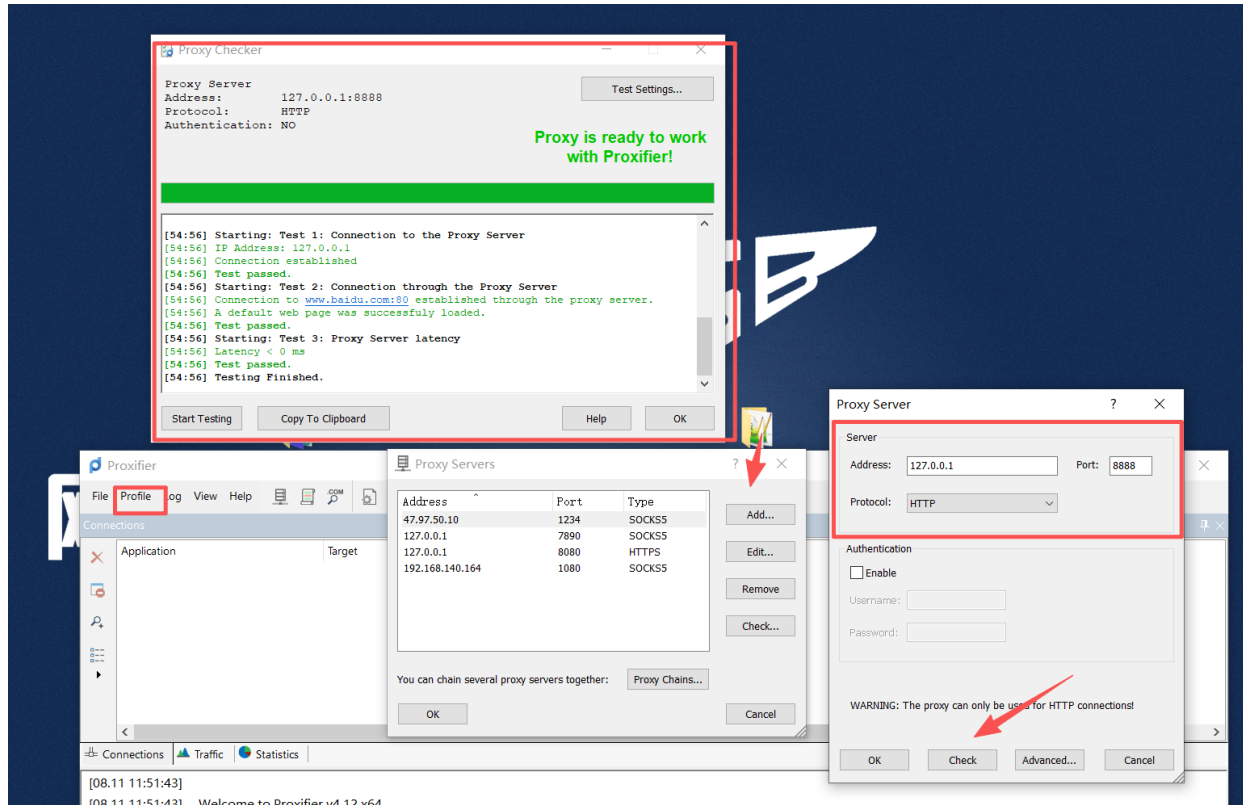
Port: 8888

Protocol: 选HTTP, 别选HTTPS

填完点右边的Check按钮, 测试代理连通性。

提示Proxy is ready to work with Proxifier就说明没问题, 点OK保存。

如果测试失败, 回去看Fiddler有没有启动, 端口是不是8888, 有没有开允许远程连接。



6.3 代理规则配置（顺序决定生死）

打开 配置文件 → 代理规则, 默认会有一条Default规则, 我们再加两条, 总共三条, 按下面的顺序从上到下排。

第一条: Burp相关进程直连（最高优先级）

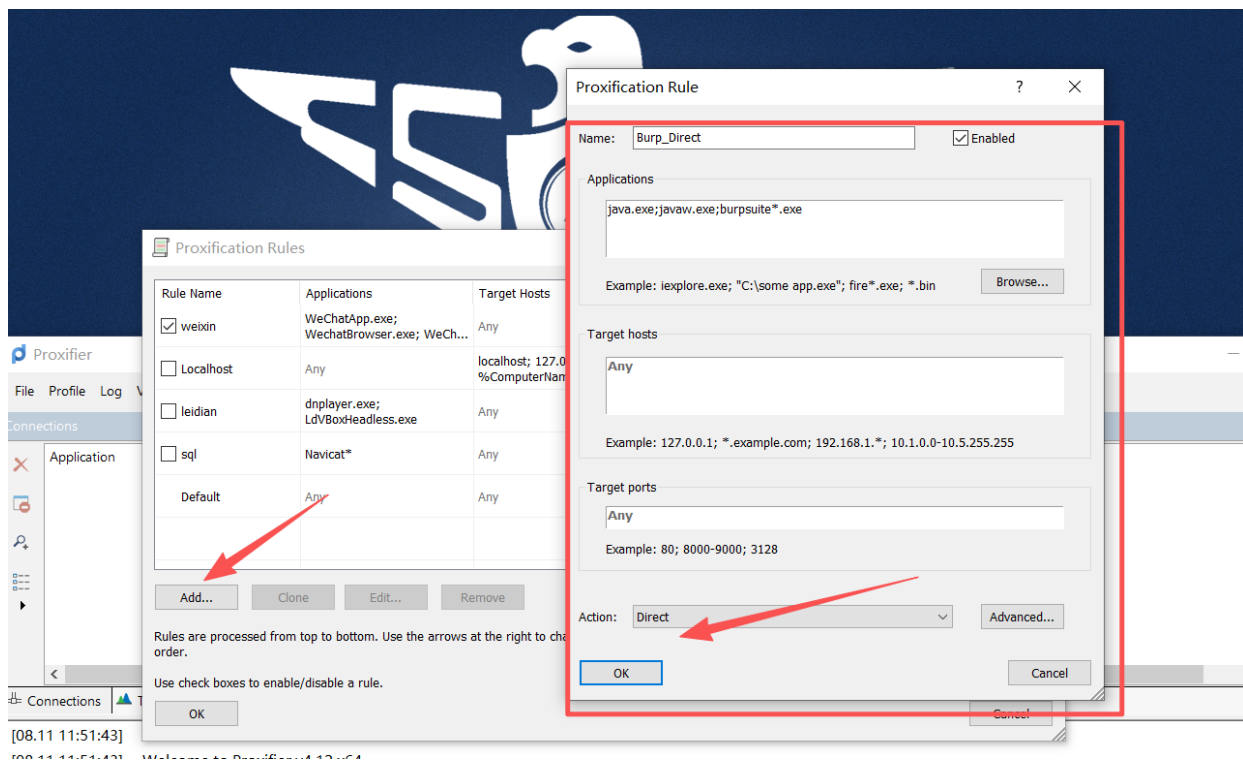
名称: Burp_Direct

应用程序: java.exe;javaw.exe;burpsuite*.exe

目标地址、目标端口都留空（匹配所有）

动作: Direct（直连, 不走代理）

Burp本身是Java程序, 它向外发请求的时候, 如果也被Proxifier劫持, 就会形成 Burp→Fiddler→Burp 的死循环, 流量无限转发, 直接卡死网络。所以这条规则必须放最上面, 让Burp自己的流量直接走网卡, 不进代理链路。



第二条：微信全进程强制走代理（核心抓包规则）

名称：WeChat_Mini_Proxy

应用程序：weChat.exe; weChatAppEx.exe; weChatBrowser.exe

目标地址、目标端口留空

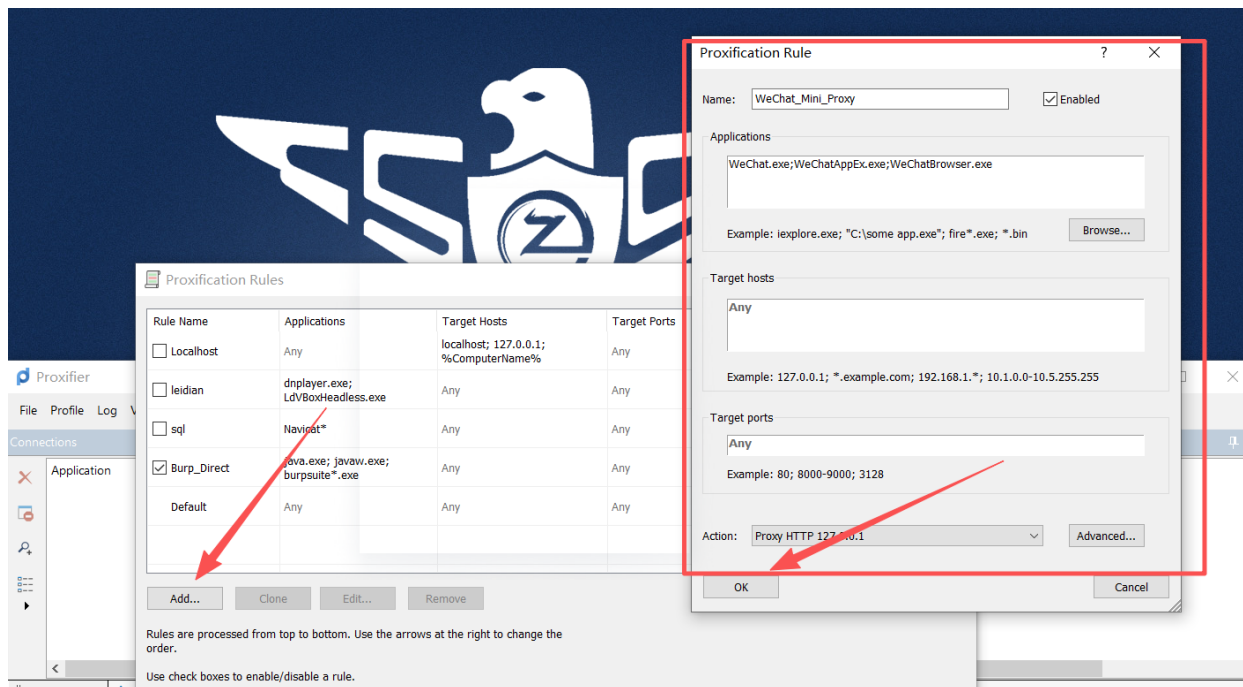
动作：选我们刚加的 HTTP 127.0.0.1:8888 代理

三个进程一个都不能少：

weChat.exe是微信主进程，负责登录、基础通信

weChatAppEx.exe是小程序独立进程，我们要抓的核心流量都在这

weChatBrowser.exe是微信内置浏览器进程，公众号、H5页面的流量走这



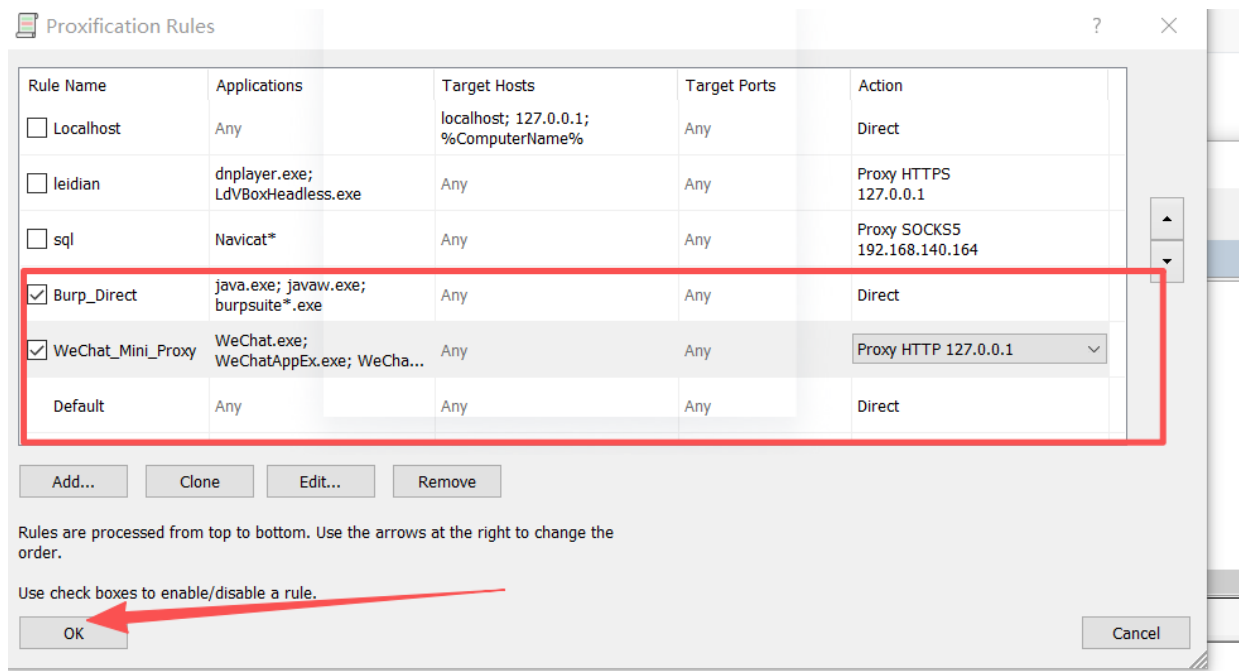
第三条：Default默认规则

动作: Direct

所有其他程序的流量全部直连，不经过代理。不然你浏览器、QQ、游戏全都走代理，不仅卡，还容易把无关流量混进抓包结果里。

配完之后用右边的上下箭头调整顺序，确保顺序是：

Burp_Direct → WeChat_Mini_Proxy → Default
点确定保存规则。



配图：Proxifier代理规则优先级示意图

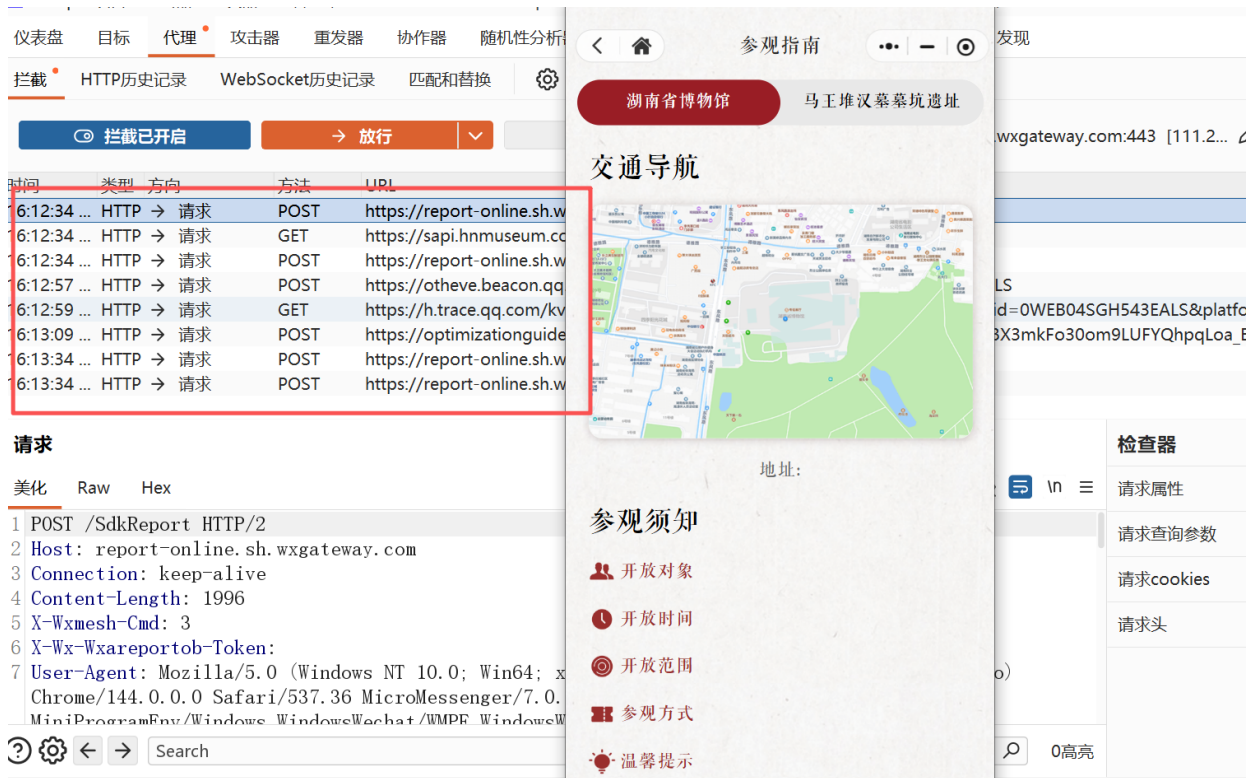
（图示说明：三条规则从上到下排列，用箭头标注匹配顺序，标注每条规则的作用与优先级，重点标红「Burp直连规则优先级最高」）

如果有多开微信的需求，Proxifier会劫持所有微信进程。只想抓单个账号的话，可以临时在任务管理器里找到对应进程的PID，把规则里的进程名换成带PID的精确匹配。不过每次重启微信PID都会变，只适合临时测试用。

0x07 全链路验证与抓包实操

所有配置都做完了，按顺序启动工具：

启动Burp Suite，确认8080端口监听正常
启动Fiddler，确认8888端口正常
启动Proxifier，确认代理规则加载完成
最后打开PC微信，随便点开一个业务接口多的小程序，比如电商、生活服务类



7.1 逐层验证链路通不通

先看Proxifier的日志窗口。

正常情况下，你点开小程序的瞬间，Proxifier里会刷出大量WeChatAppEx.exe的连接记录，状态是绿色的，代理列显示127.0.0.1:8888。

如果日志是空的，说明规则没匹配到，回去检查进程名有没有写错，规则顺序对不对，有没有开管理员权限。

再看Fiddler的会话列表。

Proxifier有流量的话，Fiddler里会出现对应的请求，域名包括微信基础接口和小程序业务域名。

如果Fiddler里的HTTPS会话是红色的，提示证书错误，回去检查Fiddler的HTTPS解密有没有开，根证书有没有装对。

启动Burp Suite，确认8080端口监听正常

启动Fiddler，确认8888端口正常

启动Proxifier，确认代理规则加载完成

最后打开PC微信，随便点开一个业务接口多的小程序，比如电商、生活服务类

#	Result	Protocol	Host	URL	Body	Caching	Content-Type	Process	Comments	Custom
1...	200	HTTP	Tunnel to	adtrack.xiaojukeji.com:443	747			wechatappx:17732		
1...	200	HTTPS	adtrack.xiaojukeji.com	/trackx?eid=5&bus=256&...	2		text/plain; ...	wechatappx:17732		
1...	200	HTTPS	adtrack.xiaojukeji.com	/trackx?eid=5&lc=&phb=...	2		text/plain; ...	wechatappx:17732		
1...	200	HTTPS	adtrack.xiaojukeji.com	/trackx?eid=5&egt=engin...	2		text/plain; ...	wechatappx:17732		
1...	200	HTTP	Tunnel to	adtrack.xiaojukeji.com:443	0					
1...	200	HTTPS	adtrack.xiaojukeji.com	/trackx?eid=5&openid=&...	2		text/plain; ...	wechatappx:17732		
1...	200	HTTP	Tunnel to	adtrack.xiaojukeji.com:443	0					
1...	200	HTTPS	adtrack.xiaojukeji.com	/trackx?eid=5&scst=6795...	2		text/plain; ...	wechatappx:17732		
1...	200	HTTP	Tunnel to	adtrack.xiaojukeji.com:443	0					
1...	200	HTTPS	adtrack.xiaojukeji.com	/trackx?eid=5&ded=0&oa...	2		text/plain; ...	wechatappx:17732		
1...	200	HTTP	Tunnel to	adtrack.xiaojukeji.com:443	0					
1...	200	HTTPS	adtrack.xiaojukeji.com	/trackx?eid=5&xprod=&sd...	2		text/plain; ...	wechatappx:17732		
1...	200	HTTPS	adtrack.xiaojukeji.com	/trackx?eid=5&lang=zh-C...	2		text/plain; ...	wechatappx:17732		
1...	200	HTTP	Tunnel to	adtrack.xiaojukeji.com:443	0					
1...	200	HTTP	Tunnel to	adtrack.xiaojukeji.com:443	0					
1...	200	HTTPS	ut-static.udache.com	/webx/ut/Vaze2f8pwly4...	67,373	max-ag...	image/png	wechatappx:17732		
1...	200	HTTPS	ut-static.udache.com	/webx/ut/3bgknlF_45DrK...	444	max-ag...	image/png	wechatappx:17732		
1...	200	HTTP	Tunnel to	ut-static.udache.com:443	0					
1...	200	HTTP	Tunnel to	ut-static.udache.com:443	0					
1...	200	HTTPS	ut-static.udache.com	/webx/ut/16xrnaJk5UmXd...	254	max-ag...	image/webp	wechatappx:17732		
1...	200	HTTP	Tunnel to	ut-static.udache.com:443	0					
1...	200	HTTPS	ut-static.udache.com	/webx/ut/ZXJgibEq3G5iU...	163,992	max-ag...	binary/octe...	wechatappx:17732		
1...	200	HTTPS	ut-static.udache.com	/webx/ut/d98D3ZGEMMS9...	4,392	max-ag...	font/ttf	wechatappx:17732		
1...	200	HTTP	Tunnel to	ut-static.udache.com:443	0					
1...	200	HTTP	Tunnel to	ut-static.udache.com:443	0					
1...	200	HTTPS	omgup.xiaojukeji.com	/api/realtime/stat/mini?e=...	14		text/plain	wechatappx:17732		
1...	200	HTTP	Tunnel to	omgup.xiaojukeji.com:443	0					
1...	200	HTTPS	ut.xiaojukeji.com	/ut/kappa/api/owner/getN...	87		application/...	wechatappx:17732		
1...	200	HTTPS	adtrack.xiaojukeji.com	/trackx?eid=7&oam5=&fs...	2		text/plain; ...	wechatappx:17732		
1...	200	HTTP	Tunnel to	ut.xiaojukeji.com:443	0					
1...	200	HTTP	Tunnel to	adtrack.xiaojukeji.com:443	0					
1...	200	HTTP	pr.map.qq.com:443		0			wechatappx:17732		
1...	200	HTTPS	pr.map.qq.com	/pingd?appid=jsapi_v3&o...	43	no-store	image/gif	wechatappx:17732		
1...	200	HTTPS	omgup.xiaojukeji.com	/api/minicompress/x?coun...	157		text/plain	wechatappx:17732		
1...	200	HTTP	Tunnel to	omgup.xiaojukeji.com:443	0					
1...	200	HTTP	report-online.sh.wxgatew...		0			wechatappx:17732		
1...	304	HTTP	mmbiz.qpic.cn	/sz_mmbiz_png/lbQtPRW...	0	max-ag...	image/webp	wechatappx:17732		
1...	200	HTTPS	report-online.sh.wx...	/SdkReport	2		application/...	wechatappx:17732		
1...	200	HTTP	Tunnel to	accounts.google.com:443	0			wemeetapp:7568		

最后看Burp Suite。

切到Proxy → HTTP history，里面会同步出现Fiddler转发过来的请求，都是解密后的明文HTTPS，请求头、请求体、返回值全都能看到。

如果Fiddler有流量，Burp里是空的，回去查Fiddler的转发脚本有没有写对，Burp的8080端口有没有正常监听，透明代理开没开。

拦截	HTTP历史记录	WebSocket历史记录	匹配和替换	代理设置
过滤器设置:Hiding CSS and image content; 隐藏指定的扩展名				
Search				
Host	方法	URL	参数	已编... 状态码 长度 MIME... 扩展 标题 说明 TLS IP Cookies
https://omgup.xiaojukeji.com	POST	/api/realtime/stat/mini...	✓	200 186 文本
https://ut.xiaojukeji.com	POST	/ut/kappa/api/owner/g...	✓	200 564 JSON
https://adtrack.xiaojukeji.com	GET	/trackx?eid=7&oam5=...	✓	200 453 文本
https://omgup.xiaojukeji.com	POST	/api/minicompress/x?c...	✓	200 330 文本
请求				
美化 Raw Hex	美化 Raw Hex 页面渲染			
1 GET /trackx?eid=7&oam5=&fs=0&llc=&exp=&deid=&omega_id=&st=1786436242778&xpsid=&schg=626587&ctyid=0&fbk=others&xpsid_root=&it=1&aki=0&ori=0&ht=0&ant=1358&idsp=1&dut=__DURATION__&jump_type=&res=989&openid=&st=0&sat=__STARTTIME__&xpage=&epr=&cit=17&ft=1&ent=__ENDTIME__&apv=6.2.5&cost=0&ddfp=&lng=0.000000&wkt=e10152bc6a7ada9296e2f4eb6505d0e6&sbk=240722&sch=533309&cm=0&slide_true=1&sg=05cc0a9f92ed2cfcb2d876cf6a676c9e223199d262024b7cc584f7dc603a62&oaaid=&wd=0&xprod=&idfa=&uid=299068127845347&sn=&wtp=nil&teid=1451&act=2723979&bt=0&grp=&ded=0&rt=1786436242740&fit=1786436242778&idm5=&ct=0&tkn=BFmey07uUkyHxWk1Z28RoSYmBuFhZCnupx1MX0oeIMM0yzEKAKEMRuG7_HVYkkyk-Q46ipYjKBYLXt3mcL2Pb4Dk1HQjTcGYQrKCbOh1JmTMBU1w1PPFRcw81rbVoptmqBauqJbJPaSNMfcbbhBtKCDvqvOf1fv931DNznoT7X_e29AMF8eG9s6gFCE-UnL8AAAD_w...				
1 HTTP/2 200 OK				
2 Date: Tue, 11 Aug 2026 08:17:23 GMT				
3 Content-Type: text/plain; charset=utf-8				
4 Content-Length: 2				
5 Access-Control-Allow-Headers: Didi-Header-Hint-Content				
6 Access-Control-Allow-Headers: X-Prod-Session-Id				
7 Access-Control-Allow-Headers: X-Prod-Key				
8 Access-Control-Allow-Headers: X-Act-Key				
9 Access-Control-Allow-Methods: POST				
10 Access-Control-Allow-Methods: GET				
11 Access-Control-Allow-Origin: *				
12 X-Dfe-Req-Ts: 1786436243.972, 1786436243.976				
13				
14 {}				

7.2 基础抓包操作

链路通了之后，常规的安全测试操作都能正常用：

拦截请求：Proxy页面切到Intercept，点Intercept is on，小程序发的请求会被Burp截停，可以改参数、改请求头再发出去

重放测试：选中历史请求，右键Send to Repeater，改参数直接重放，看接口返回逻辑

流量过滤：Burp的Filter栏可以按域名、状态码过滤，只看目标业务的接口，不用被微信基础接口的流量淹没

爆破测试：Send to Intruder，对参数做字典爆破，测试越权、遍历类漏洞

抓包的时候注意区分微信基础接口和业务接口。api.weixin.qq.com、open.weixin.qq.com这类域名是微信官方的开放平台接口，一般不用关注。重点看小程序所属企业自己的业务域名，通常是备案过的企业域名，所有业务逻辑请求都走这些域名。

0x08 移动端微信小程序抓包方案

PC端抓包适合快速调试，真要测线上全量逻辑、复现用户端问题，还是得用手机端。移动端抓包的核心障碍是证书信任机制，尤其是安卓7.0之后，系统不再默认信任用户安装的证书。

8.1 基础环境准备

手机和电脑连同一个局域网WiFi，查一下电脑的内网IP。Windows按Win+R输cmd，打开命令提示符，输ipconfig，看IPv4地址，一般是192.168.x.x开头。

先把Burp的监听地址改成All interfaces，不然手机连不上。在Burp的Proxy Listeners里，把8080的监听地址改成All interfaces，保存。

手机端代理可以直接连Burp的8080端口，也可以连Fiddler的8888端口，推荐走Fiddler，兼容性更好。

8.2 iOS端配置

iOS的证书逻辑比较简单，只要正确安装并信任根证书，就能正常解密绝大多数小程序的HTTPS流量。

手机WiFi设置，点当前连接的WiFi右边的感叹号，拉到最下面，代理选「手动」

服务器填电脑的内网IP，端口填8888

打开Safari浏览器，访问 http://电脑IP:8888，下载Fiddler或者Burp的证书描述文件

打开设置 → 通用 → VPN与设备管理，找到刚下载的描述文件，完成安装

再打开设置 → 通用 → 关于本机 → 证书信任设置，找到对应根证书，把后面的开关打开

做完这五步，打开微信小程序，就能在电脑端看到解密后的流量。iOS系统对证书的管控比较严格，只要是系统信任的根证书，微信WebView都会认可，基本不会出现解密失败的问题。如果碰到风控严格的小程序提示网络异常，就是开启了SSL Pinning，需要用后面的绕过方案。

8.3 安卓端配置（重点：系统证书）

安卓7.0开始，应用默认只信任系统分区的根证书，用户自己安装的证书不算数。普通安装证书的方法，抓浏览器没问题，抓微信小程序直接解密失败。

分两种方案，Root和免Root，根据自己的设备选择。

方案A: Root手机, 安装证书到系统分区

这是稳定性最高的方案, 适配所有安卓版本和微信版本。

先把Burp的DER证书转成PEM格式, 并且计算hash重命名。电脑端用openssl工具转换, 没有的话先安装OpenSSL:

```
转换证书格式 DER转PEM
openssl x509 -inform DER -in BurpCA.cer -out BurpCA.pem

计算证书hash值, 作为系统证书的文件名
openssl x509 -inform PEM -subject_hash_old -in BurpCA.pem | head -1
```

执行完第二条命令会输出一串8位的hash值, 比如9a5ba575。

执行完第二条命令会输出一串8位的hash值, 比如9a5ba575。

2. 把证书文件重命名为 9a5ba575.0 (hash值加.0后缀, 注意是数字0不是字母O)
3. 用adb把证书推到系统证书目录, 执行以下命令:

```
adb root
adb remount
adb push 9a5ba575.0 /system/etc/security/cacerts/
adb shell chmod 644 /system/etc/security/cacerts/9a5ba575.0
adb reboot
```

手机重启之后, Burp的证书就变成系统信任的根证书了, 微信小程序的HTTPS流量可以直接解密。可以在设置 → 安全 → 信任的凭据 → 系统里, 确认证书是否安装成功。

方案B: 免Root方案

不想Root的话, 用虚拟机或者框架绕过, 适合临时测试。

下载VMOS Pro、光速虚拟机这类安卓虚拟机, 在虚拟机里安装微信。虚拟机自带Root权限, 按上面的方法把证书装到系统分区即可

或者用太极、LSPatch框架, 给微信装上JustTrustMe模块, 直接跳过应用层的证书校验

免Root方案兼容性差一点, 微信版本更新之后可能失效, 而且虚拟机性能有限, 适合快速验证场景。

0x09 高频踩坑排查手册

我整理了配置过程中90%的人会碰到的问题, 直接对照着修。

坑1: Proxifier完全没流量, 小程序正常加载

进程名写错了。确认任务管理器里的小程序进程是不是WeChatAppEx.exe, 有些旧版本微信叫WeChatMiniProgram.exe

没开管理员权限。Proxifier没有管理员权限的话, 劫持不到其他进程的流量

规则顺序错了。微信规则被Default盖住了, 流量直接直连走了

微信是UWP应用。微软商店下载的微信是UWP版本, 有沙箱隔离, Proxifier抓不到, 去微信官网下载桌面版

坑2：HTTPS流量显示加密，全是乱码，解密失败

证书装错位置了。Burp和Fiddler的证书都必须装到本地计算机的受信任根证书颁发机构
安卓手机没装系统证书，只装了用户证书。安卓7+以上用户证书默认不被应用信任
小程序开启了SSL Pinning，证书锁定，需要用下一节的绕过方案
浏览器缓存了证书。清空微信缓存，重启小程序再试

坑3：配置完之后微信直接断网，小程序加载不出来

死循环了。Burp的直连规则没放最上面，Burp流量又被转发回代理，无限循环
Fiddler或者Burp没启动，Proxifier把流量导去一个不存在的端口，肯定连不上
杀毒软件拦了。把安全软件临时退了再试，尤其是火绒的网络防护功能
端口被占用了。用netstat -ano命令查一下8888和8080端口有没有被其他程序占用

坑4：Fiddler有流量，Burp里什么都没有

Fiddler的转发脚本没保存，或者代码写错了。检查OnBeforeRequest里的代码，拼写有没有错
Burp的监听端口不是8080，或者监听地址不是127.0.0.1
透明代理没开。Burp接不到非标准代理格式的请求
Fiddler开了全局上游代理，和脚本冲突。把全局代理设置清空，只用脚本控制转发

坑5：小程序白屏，提示网络环境异常

微信检测到了代理环境。部分风控严的小程序会识别中间人攻击，直接拒绝服务
大概率是开启了SSL Pinning，需要用Frida hook绕过证书校验
微信版本太新，加了新的风控逻辑。降级到3.9.5左右的稳定版本再试

0x10 绕过SSL Pinning证书锁定

普通的小程序装完证书就能抓，但金融、支付、银行类的小程序，会做SSL Pinning（证书锁定）。应用里内置了服务器证书的公钥hash，中间人证书哪怕是系统信任的，只要公钥对不上，直接断开连接。
这种情况光装证书没用，得hook掉证书校验逻辑。

10.1 PC端微信绕过方案

PC端用Frida注入WeChatAppEx.exe进程，Hook系统和Chromium内核的证书校验函数。
先在电脑上装Frida环境，需要提前装好Python：

```
pip install frida-tools
```

然后新建一个pc_ssl_unpin.js的脚本文件，复制以下内容：

```
// PC端微信小程序SSL Pinning绕过脚本
(function() {
    console.log("[+] Loading SSL Pinning bypass for WeChatAppEx...");

    // Hook windows证书链校验API
    var crypt32 = Module.load('crypt32.dll');
```



```

var CertVerifyCertificateChainPolicy =
crypt32.findExportByName('CertVerifyCertificateChainPolicy');

Interceptor.attach(CertVerifyCertificateChainPolicy, {
  onLeave: function(retval) {
    // 强制返回证书校验通过
    retval.replace(0);
  }
});

// Hook WinHTTP安全选项查询
var wininet = Module.load('wininet.dll');
var InternetQueryOptionA = wininet.findExportByName('InternetQueryOptionA');

Interceptor.attach(InternetQueryOptionA, {
  onEnter: function(args) {
    // INTERNET_OPTION_SECURITY_FLAGS = 31
    // INTERNET_OPTION_CLIENT_CERT_CONTEXT = 84
    var option = args[1].toInt32();
    if (option === 31 || option === 84) {
      this.skipCheck = true;
    }
  },
  onLeave: function(retval) {
    if (this.skipCheck) {
      retval.replace(1);
    }
  }
});

// Hook Chromium网络栈证书校验
try {
  var netModule = Process.findModuleByName('net.dll');
  if (netModule) {
    var verifyFunc = Module.findExportByName('net.dll', 'CertVerifyProc');
    if (verifyFunc) {
      Interceptor.attach(verifyFunc, {
        onLeave: function(retval) {
          retval.replace(0);
        }
      });
    }
  }
} catch (e) {}

console.log("[+] SSL Pinning bypass loaded successfully");
})();

```

打开微信，随便点开一个小程序，让 weChatAppEx.exe 进程跑起来。然后开命令行，进入脚本所在目录执行：

```
frida -n weChatAppEx.exe -l pc_ssl_unpin.js
```

注入成功之后，命令行会提示加载成功。刷新小程序，原本白屏或者报错的页面就能正常加载，Burp里也能看到解密后的明文流量。

如果有多个小程序进程，Frida会注入第一个匹配的进程。要注入所有进程的话，加个--attach-all参数即可。

10.2 安卓端绕过方案

安卓端绕过SSL Pinning的方案更多，根据手机环境选就行。

最省心的是用Xposed模块。Root手机装LSPosed，下载JustTrustMe++模块，作用域勾选微信，重启微信就生效，能绕过绝大多数应用的证书锁定，包括微信小程序。

如果不想装Xposed框架，直接用Frida注入也可以。新建android_ssl_unpin.js脚本：

```
// 安卓微信小程序通用SSL Unpin脚本
Java.perform(function() {
    console.log("[+] Starting Android SSL unpin...");

    // Hook系统默认X509TrustManager
    var TrustManager = Java.use('javax.net.ssl.X509TrustManager');
    TrustManager.checkServerTrusted.implementation = function(chain, authType) {
        return;
    };
    TrustManager.checkClientTrusted.implementation = function(chain, authType) {
        return;
    };

    // Hook WebView SSL错误处理
    var WebViewClient = Java.use('android.webkit.WebViewClient');
    WebViewClient.onReceivedSslError.implementation = function(view, handler, error)
    {
        handler.proceed();
    };

    // Hook OkHttp证书锁定
    try {
        var CertificatePinner = Java.use('okhttp3.CertificatePinner');
        CertificatePinner.check.overload('java.lang.String',
        'java.util.List').implementation = function(hostname, certificates) {
            return;
        };
    } catch (e) {}

    // Hook微信X5内核证书校验
    try {
        var X5WebView = Java.use('com.tencent.smtt.sdk.WebviewClient');
        X5WebView.onReceivedSslError.implementation = function(view, handler, error)
        {
            handler.proceed();
        };
    } catch (e) {}
});
```

```
console.log("[+] Android SSL unpin success");
});
```

手机开启USB调试，连接电脑，执行命令注入微信进程：

```
frida -U -f com.tencent.mm -l android_ssl_unpin.js --no-pause
```

微信会自动重启，注入成功之后打开小程序，就能正常抓包。

HTTPS中间人解密与SSL Pinning绕过原理图

（分三栏展示，左栏正常HTTPS链路，服务器证书→客户端校验通过；中栏中间人攻击链路，代理证书被客户端Pinning校验拒绝；右栏绕过Pinning链路，Hook校验函数强制返回通过，中间人解密成功）

0x11 抓包后的实战应用场景

抓包只是手段，最终要落地到实际工作里。这套链路可以覆盖绝大多数小程序的测试场景。

11.1 接口安全测试

最常用的场景。抓出小程序的业务接口之后，测越权、测注入、测参数篡改。比如用户信息接口改个userID能不能拿到别人的数据，支付接口改个金额能不能下单成功。

Burp的Repeater和Intruder可以直接用，和测普通Web接口没有区别。小程序的前端校验都是摆设，所有安全漏洞最终都要落到服务端接口层面验证，抓包是必经之路。

11.2 加密参数逆向

很多小程序的请求参数是加密的，比如sign签名字段、请求体AES加密。抓包拿到明文请求和加密后的密文，配合反编译小程序代码，逆向加密逻辑，写脚本批量生成加密参数。

Proxifier+Fiddler的链路稳定性高，不会漏掉请求，适合做全量的流量分析，找加密规律。碰到复杂的国密、自定义加密算法，还可以配合Frida hook加密函数，直接拿明文和密钥。

11.3 业务逻辑漏洞挖掘

比如优惠券领取、抽奖、签到这类接口，测重放能不能重复领取，测参数篡改能不能修改中奖结果。很多小程序前端限制了抽奖次数，服务端没做校验，抓包重放就能无限抽奖。

还有订单状态修改、积分篡改这类逻辑漏洞，都需要抓包之后逐一测试接口的校验逻辑。

11.4 前端性能分析

抓包看每个接口的响应时间、资源加载顺序，优化小程序首屏加载速度。Fiddler自带统计功能，可以直接看每个请求的耗时、带宽占用。

还可以模拟弱网环境，测试小程序在网络差的时候的表现。Fiddler可以设置限速，模拟2G、3G、弱网环境，看页面会不会出现加载异常、逻辑错乱。